Embedded C programming





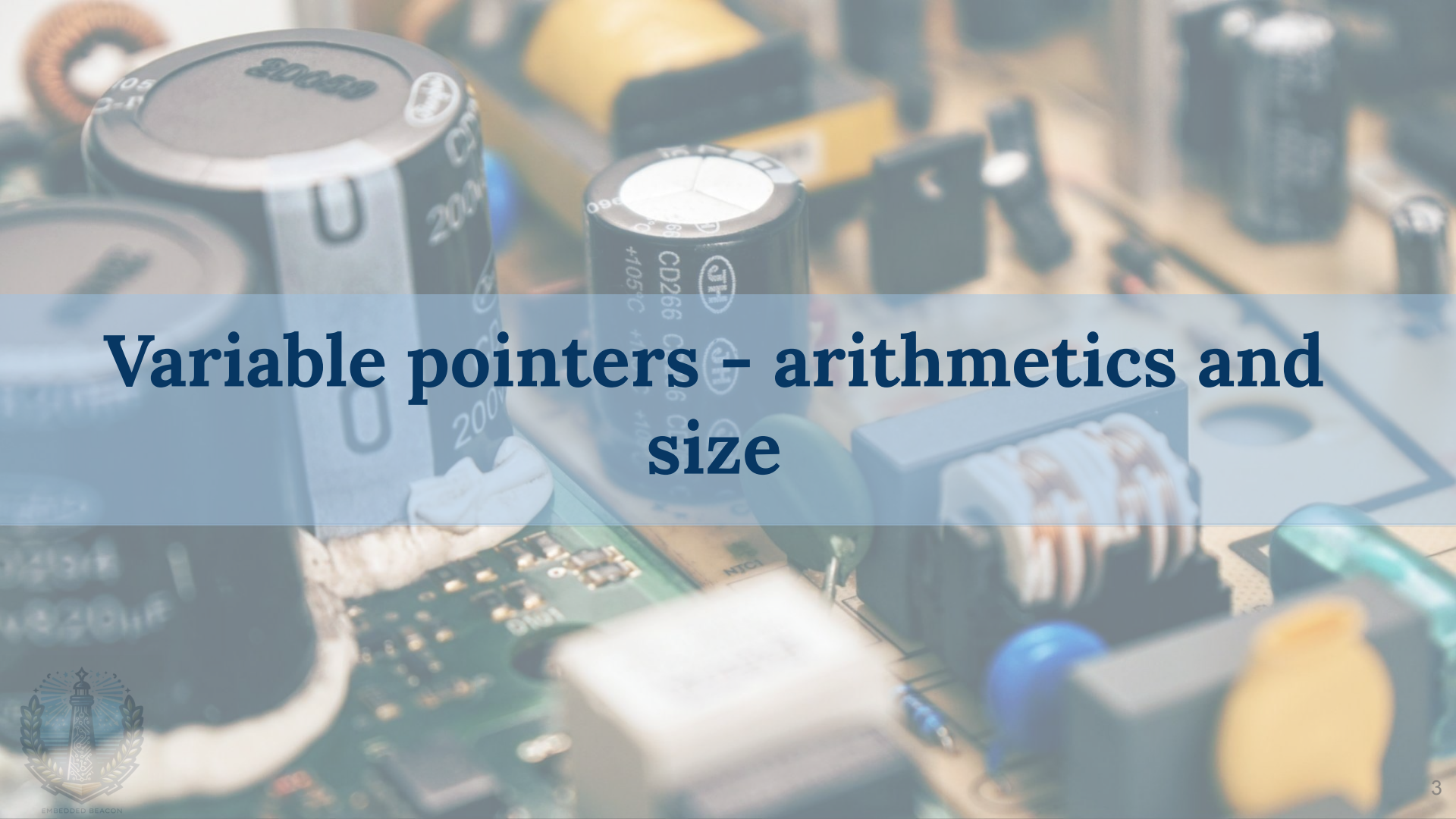


Hello!

We are Embedded Beacon

Follow us for high quality Embedded contents





Variable pointers - arithmetics and size

Pointers

In embedded systems, understanding pointer arithmetic and sizes is crucial for handling memory efficiently and interfacing with hardware.

Embedded systems often have constrained memory, and direct manipulation of pointers allows precise control over memory regions, registers, and data structures.

Example 1: Buffer Management in Embedded Systems Using Pointer Arithmetic

Consider an embedded system where we need to manage a buffer used for serial communication (UART).

We might need to access specific elements within the buffer, which can be efficiently done using pointer arithmetic.

```c
#include <stdio.h>
#include <stdint.h>

#define BUFFER_SIZE 5

volatile uint8_t uart_buffer[BUFFER_SIZE]; // UART buffer for data storage

// Function to fill the buffer with data
void fill_buffer() {
    volatile uint8_t *buffer_ptr = uart_buffer; // Pointer to the beginning of the buffer
    for (uint16_t i = 0; i < BUFFER_SIZE; i++) {
        *(buffer_ptr + i) = i % 256; // Fill the buffer using pointer arithmetic
    }
}

// Function to print buffer contents
void print_buffer_contents() {
    volatile uint8_t *buffer_ptr = uart_buffer; // Pointer to the buffer
    for (uint16_t i = 0; i < BUFFER_SIZE; i++) {
        uint8_t value = *(buffer_ptr + i); // Access elements using pointer arithmetic
        printf("Buffer[%u] = %u\n", i, value); // Print value to console
    }
}

int main() {
    // Fill the buffer with data
    fill_buffer();

    // Print the contents of the buffer
    print_buffer_contents();

    return 0;
}
```

Output

```
Buffer[0] = 0
Buffer[1] = 1
Buffer[2] = 2
Buffer[3] = 3
Buffer[4] = 4
```

Explanation

1. **Pointer Arithmetic:**
   - buffer_ptr + i increments the pointer by i elements, with each increment corresponding to the size of the data type (uint8_t in this case).
   - The expression *(buffer_ptr + i) accesses the value at the address computed by the pointer arithmetic.
2. **Why Use Pointer Arithmetic:**
   - In embedded systems, accessing elements with *(buffer_ptr + i) instead of uart_buffer[i] is sometimes preferable, as it provides more explicit control over memory access patterns. This can be particularly useful when dealing with DMA, ring buffers, or hardware buffers.



Example 2: Handling Different Data Types and Sizes with Pointer Arithmetic

Pointer arithmetic in C is affected by the type of the pointer, which determines the size of the memory being addressed. In embedded systems, you often need to manipulate registers and data structures of different sizes.

Suppose we have a memory-mapped region representing different hardware registers, where some registers are 8-bit and others are 16-bit:

EMBEDDED BEACON

```c
#include <stdint.h>
#include <stdio.h> // For demonstration purposes, this will simulate register changes

#define REGISTER_BASE_ADDRESS 0x20000000U

// Simulated memory for the purpose of this example
uint8_t simulated_memory[256] = {0};

// Register pointers pointing to specific memory locations
volatile uint8_t *reg8_ptr = (volatile uint8_t *)(simulated_memory);               // 8-bit register pointer
volatile uint16_t *reg16_ptr = (volatile uint16_t *)(simulated_memory + 4);        // 16-bit register pointer

void access_registers() {
    // Access and modify 8-bit register
    *reg8_ptr = 0xFF; // Set the 8-bit register to 0xFF
    printf("8-bit register at offset 0: 0x%02X\n", *reg8_ptr); // Print the value for verification
    reg8_ptr += 2;    // Move to the next 8-bit register (pointer arithmetic)

    // Access and modify 16-bit register
    *reg16_ptr = 0xABCD; // Set the 16-bit register to 0xABCD
    printf("16-bit register at offset 4: 0x%04X\n", *reg16_ptr); // Print the value for verification
    reg16_ptr += 1;      // Move to the next 16-bit register (pointer arithmetic)
}

int main() {
    // Access registers using pointers
    access_registers();

    // Print the updated memory values to simulate register changes
    printf("Memory content at offset 0: 0x%02X\n", simulated_memory[0]);
    printf("Memory content at offset 4: 0x%02X 0x%02X\n", simulated_memory[4], simulated_memory[5]);

    return 0;
}
```

Output

```
8-bit register at offset 0: 0xFF
16-bit register at offset 4: 0xABCD
Memory content at offset 0: 0xFF
Memory content at offset 4: 0xCD 0xAB
```

Explanation

Simulated Memory Array:

- Since we're working in a simulated environment and not on an actual embedded system, uint8_t simulated_memory[256] is used to simulate memory locations.
- In real embedded systems, the pointers would point directly to memory-mapped register addresses instead of an array.

Pointer Declaration:

- volatile uint8_t *reg8_ptr = (volatile uint8_t *)(simulated_memory) points to the base address of the simulated memory.
- volatile uint16_t *reg16_ptr = (volatile uint16_t *)(simulated_memory + 4) points to an address offset by 4 bytes to simulate a different register.

Explanation

Pointer Arithmetic:

- *reg8_ptr = 0xFF sets the 8-bit value at the register pointed by reg8_ptr.
- reg8_ptr += 2 increments the pointer by 2 bytes. Since reg8_ptr is a uint8_t * (1 byte in size), adding 2 will point to the register at offset 2.
- *reg16_ptr = 0xABCD sets the 16-bit value at the register pointed by reg16_ptr.
- reg16_ptr += 1 increments the pointer by 2 bytes since reg16_ptr is a uint16_t *.

Main Function:

- The main function calls access_registers and then prints the memory content to show how the simulated registers have changed.



Thank you